1- React Component

```
// One Component per File, Capital letter name, export at bottom
function BookCard ({ title, author, category })
  return (
<  div
    <h1> {title} </h1>
    <p> Author: {Author} </p>
    <p> Category: {Category} </p>
    </ div>
  )
}
export default BookCard
```

2- Using a Component

```
// static — hard coded values
< BookCard title = "The Jungle" author = "Zola" category = "Fiction" />
// Dynamic — From data
books.map (book => (< bookCard  key = {book.id}  // required, use id not array index
                                title = {book.title}
                                author = {book.author}
                                category = {book.category}
                    />
          ))
```

3- USE State

```
Const [Value , setValue] = useState (initialeValue)
// value = Read it
// setValue = only way to change it, triggers re-render
Const [ books, setBooks ] = useState([])   // array
Const [ loading, setLoading ] = useState ( true) // boolean
Const [ error , setError ] = useState  (null)  // null or object
Const [ search, setSearch ] = useState (' ') // string
```

4- Use Effect — Fetch on load

```
Use Effect ( () => {
        Fetch ('url')
        .then (res => res.json)
        .then (data => setBooks, (data))
        .catch (err => setError (err))
        .finally ( () => setLoading(False) )
}, [])  // [] = run once on first load only
```

5- Loading & error — always include

```
if (loading) return <div> Loading ... </div>
if (error) return < div> Error: {error.message} </div>
return <div> actual content </div>
```

6- Search Filter

```
Const [search , set search ] = useState(' ')
Const filtered = items.filter (item =>
item.name.toLowerCase().includes( search.toLowerCase() )

< input
type= "text" value = { search} onChange = {e=> setSearch(e.target.value)}
/>
 {filtered.map (item => <Card key = {item.id} name= {item.name} />)}

// App.jsx
 import { Routes, Route } from 'react-router-dom'
<Routes>
  < Route path = "/"  element = {<Home />} />
  < Route path= "/posts" element = { < Posts /> }/>
</Routes>
```

7- React Router

```
// main.jsx
import { BrowserRoute} from 'react-router-dom'
< BrowserRouter> < App/> </ BrowserRouter>

// Navbar.jsx
import { Link } from 'react-router-dom'
< Link to = "/" > Home </ Link>
< Link to = "/posts"> Posts < / Link>
```

8- Full CRUD Fetch Patterns

```
// GET

fetch ('URL') . then (res => res.json()) . then (data => setData (data) )
// POST
fetch ('url', {   method : ' POST ',
                  headers : {' Content-Type ': 'application/json'},
                  body :  JSON.stringify ( { title , body })
              } )
```

```
// PUT

fetch ('url/${id}',{method: 'PUT',
                    headers : {'Content-Type': 'application/json'},
                    body:  JSON.stringify ({title, body})
                   })

// DELETE

fetch ('url/${id}', {method: 'DELETE'})
```

9- Form with Submit (Post)

```
Function handleSubmit (e) {

 e.preventDefault ()

 fetch ('url', {

      method: 'Post',
      headers : {'Content-type': 'application/json'},
      body : JSON.stringify ({title})

      })
      .then (res => res.json())
      .then (data => {setSuccess(data)
                      setTitle ('')}

            )

       }

<Form   onSubmit= {handleSubmit}>
     <input  type="text" value = {title} onChange = {e=> setTitle (e.target.value)}/>

     <button  type = "submit "> Submit </button>
    { success && <p> Created : { success.title } </p> }

</Form>
```

10- Edit & Delete Pattern

```
const [deleted, setDeleted] = useState(false)
const [isEditing, setIsEditing] = useState(false)
const [newTitle, setNewTitle] = useState(title)

function handleDelete() {
  fetch('url/${id}', {method: 'DELETE'})
    .then(() => setDelete(true))
}

function handleEdit(e) {
  e.preventDefault()
  fetch('url/${id}', {method: 'put',
  headers: {'Content-type': 'application/json'},
  body: JSON.stringify({title: newTitle})
  })
    .then(res => res.json())
    .then(data => {setNewTitle(data.title); setIsEditing(false)})
}

if(deleted) return <p>Deleted.</p>
if(isEditing) return (
  <form onSubmit={handleEdit}>
      <input value={newTitle} onChange={e => setNewTitle(e.target.value)} />
      <button type="submit">Save</button>
      <button type="button" onClick={() => setIsEditing(false)}>Cancel</button>
  </form>
)
  return (
    <div>
      <h2>{newTitle}</h2>
      <button onClick={() => setIsEditing(true)}>Edit</button>
      <button onClick={handleDelete}>Delete</button>
    </div>
  )
```

11 - UseContext

Step 1: Create the context

```
import {createContext} from 'react'
export const UserContext = CreateContext (null)
```

Step 2: Provide it

```
import {useState} from 'react'
import {userContext} from './UserContext'
function App () {
    const [user, setUser]= useState ({name: 'lea', role: 'admin'})
    return (
    < userContext.Provider value={{user, setUser}}>
        <Navbar />
        <Routes> ... </Routes>
    </userContext.Provider>
)
}
```

Step 3: Consume it

```
import {useContext} from 'react'
import {UserContext} from '../UserContext'
Function Navbar (){
    const {user} = userContext(UserContext)
    return <nav> welcome, {user.name} </nav>
}
```

12 - Custom hooks

Step 1: Create a function that accepts 'URL' as parameter

```
import {useState, useEffect} from 'react'
function useFetch (url){
const [data, setData] = useState ([])
const [loading, setLoading] = useState(true)
const [error, setError] = useState (null)
useEffect ( () => {
  fetch (url)
  .then ( res => res.json())
  .then ( data => setData (data) )
  .catch (err => setError (err) )
  .finally ( () => setLoading (false))
}, [])
return {data, loading, error}
}
export default useFetch
```

must start with use

Step 2:

```
const { data: info, loading, error } = useFetch ('actual url')
```

the name of the variable in this file

A custom hook is a function that extracts a repeated logic so it can be writen once and reused everywhere.